

Лабораторная работа

Выполнил Тараканов Борис, 1032259220

РЕЛЯЦИОННЫЕ БД. ПОСТРОЕНИЕ ER-ДИАГРАММЫ. СОЗДАНИЕ БАЗЫ ДАННЫХ И ТАБЛИЦ

Цель работы: получение практических навыков построения ER-диаграмм для реляционной БД, а также получение практических навыков работы с языком SQL.

Инструменты: draw.io – свободное ПР для создания различных диаграмм; консольное приложение с утилитой ssh для возможности подключения к удаленному серверу (например putty, mobaXterm, gitbash).

1. Теоретический раздел

1.1 Реляционные базы данных

Реляционные базы данных представляют собой базы данных, которые используются для хранения и предоставления доступа к взаимосвязанным элементам информации. Реляционные базы данных основаны на реляционной модели — интуитивно понятном, наглядном табличном способе представления данных. Каждая строка, содержащая в таблице такой базы данных, представляет собой запись с уникальным идентификатором, который называют ключом. Столбцы таблицы имеют атрибуты данных, а каждая запись обычно содержит значение для каждого атрибута, что дает возможность легко устанавливать взаимосвязь между элементами данных.

Реляционная модель

В первых базах данных данные каждого приложения хранились в отдельной уникальной структуре. Если разработчик хотел создать приложение для использования таких данных, он должен был хорошо знать конкретную структуру, чтобы найти необходимые данные. Такой метод организации был неэффективен, сложен в обслуживании и затруднял оптимизацию эффективности приложений. Реляционная модель была разработана, чтобы устранить потребность в использовании разнообразных структур данных.

Она обеспечила стандартный способ представления данных и отправки запросов, которые могли быть использованы в любых приложениях. Разработчики уяснили, что таблицы являются ключевым преимуществом реляционных баз данных, так как обеспечивают интуитивно понятный,

эффективный и гибкий способ хранения структурированной информации и получения к ней доступа.

Со временем, когда разработчики стали использовать язык структурированных запросов (SQL) для записи данных в базу и отправки запросов, стало очевидным и другое преимущество реляционной модели. Вот уже на протяжении многих лет SQL широко используется в качестве языка запросов в базах данных. Он основан на алгоритмах реляционной алгебры и четкой математической структуре, что обеспечивает простоту и эффективность при оптимизации любых запросов к базе данных. Для сравнения: при использовании других подходов приходится создавать отдельные, уникальные запросы.



Рис. 1 Пример таблицы реляционной БД

Ключи

Когда речь идет о реляционной базе данных, ключи играют критическую роль в организации правильных и эффективных связей между таблицами. Основной задачей этого механизма является обеспечение целостных связей, которые помогают избежать аномалий и несогласованностей данных.

Первичный ключ (PK – primary key) – уникальный идентификатор записи в таблице. Он необходим для обеспечения уникальности каждой строки в конкретной таблице.

Внешний ключ (FK – foreign key) – ссылка на первичный ключ другой таблицы. Благодаря этому ходу обеспечивается взаимосвязь между двумя таблицами и поддерживается целостность данных.

Связь – благодаря внешним ключам записи в одной таблице могут ссылаться на записи другой таблицы, создавая структурированную и связную систему данных.

Основные характеристики реляционных баз данных

Множество сущностей. Объекты со строго определенным набором атрибутов, с помощью которых они связываются между собой, формируют понятную и простую для восприятия структуру.

Табличный форма. Такой формат гарантирует высокий уровень структурированности с жесткими логическими взаимосвязями, минимальный уровень избыточности данных, их согласованность и целостность.

Язык SQL. SQL является стандартизированным средством общения пользователя с базой данных. Он очень формальный, что делает его удобным и простым в изучении. SQL гарантирует точный результат даже при сложном многоуровневом запросе.

Масштабирование по вертикали. Реляционные базы данных хорошо масштабируются по вертикали. Но это значит, что по мере накопления информации в какой-то момент ее обработка потребует больших аппаратных ресурсов и финансовых затрат.

Масштабирование по горизонтали. Горизонтальное масштабирование, подразумевающее распределение таблиц данных по множеству серверов, является слабой стороной реляционных баз данных. С разрастанием системы появляются задержки в обновлении данных. В какие-то моменты нарушается принцип целостности данных, что может негативно отразиться на пользовательском опыте.

Наличие требований к параметрам данных. Реляционные базы данных умеют работать только со структурированными данными. Но современный цифровой мир полон неструктурированных данных (например, фото и видео), к которым нельзя применять принципы реляционной модели.

Применение реляционных баз данных

Реляционные базы данных — самые распространенные базы данных в мире. Компании самого разного размера и профиля используют их для обслуживания своих информационных систем. Такой подход удобен, чтобы:

- отслеживать торговые транзакции в интернете;
- обрабатывать критически важные данные банковских клиентов;
- хранить и обслуживать картотеки на промышленных предприятиях;
- вести учет в медицине и образовании.

То есть реляционные базы данных применимы везде, где важнейшим приоритетом является целостность и безопасность данных и где встречаются высокоструктурированные данные, соответствующие строгой, предсказуемой и предопределенной схеме.

1.2 ER-диаграммы

Схема «сущность-связь» (также ERD или ER-диаграмма) — это разновидность блок-схемы, где показано, как разные «сущности» (люди, объекты, концепции и так далее) связаны между собой внутри системы. ER-диаграммы чаще всего применяются для проектирования и отладки реляционных баз данных в сфере образования, исследования и разработки программного обеспечения и информационных систем для бизнеса. ER-диаграммы (или ER-модели) полагаются на стандартный набор символов, включая прямоугольники, ромбы, овалы и соединительные линии, для отображения сущностей, их атрибутов и связей. Эти диаграммы устроены по тому же принципу, что и грамматические структуры: сущности выполняют роль существительных, а связи — глаголов.

В ER-моделях и моделях данных обычно выделяют до трех уровней детализации:

- *Концептуальная модель данных.* Схема наивысшего уровня с минимальным количеством подробностей. Достоинство этого подхода заключается в возможности отобразить общую структуру модели и всю архитектуру системы. Менее масштабные системы могут обойтись и без этой модели. В этом случае можно сразу переходить к логической модели.
- *Логическая модель данных.* Содержит более подробную информацию, нежели концептуальная модель. На этом уровне определяются более подробные операционные и транзакционные сущности. Логическая модель не зависит от технологии, в которой она будет применяться. На этом уровне детализируют данные из концептуальной модели: к сущностям добавляют характеристики — атрибуты. Например, на логическом уровне описывают характеристики сущности «Транспорт»: марка и модель автомобиля, количество лошадиных сил, пробег, грузоподъемность.
- *Физическая модель данных.* На основе каждой логической модели данных можно составить одну или две физических модели. В последних должно присутствовать достаточно технических подробностей для составления и внедрения самой базы данных. На этом уровне

описывают, как будет организована работа с данными: выбирают тип базы, её содержание и где данные будут хранить. Например, выбирают реляционный тип базы данных и [СУБД](#) для работы с ней, перечисляют таблицы в базе и определяют, что она будет храниться на внутреннем сервере компании.

В модели есть три типа связей:

- **«Один-к-одному»** — один экземпляр сущности связан только с одним экземпляром другой сущности. Например, пассажир рейса и его место в самолете.
- **«Один-ко-многим»** — один экземпляр сущности связан со множеством экземпляров другой сущности. Например, у одного пассажира может быть несколько единиц багажа, при этом каждая единица багажа может быть связана только с одним пассажиром.
- **«Многие-ко-многим»** — множество экземпляров одной сущности связаны со множеством экземпляров другой сущности. Например, аэропорт обслуживает несколько авиакомпаний. При этом каждая авиакомпания может обслуживаться в нескольких аэропортах.

ER-диаграммы — «родственники» схем структуры данных (DSD – data structure diagramm), где вместо связей между самими сущностями отображаются отношения между элементами внутри них. ER-диаграммы часто используются в сочетании с диаграммами DFD (data flow diagramm), которые схематично показывают движение потоков информации в рамках процесса или системы.

Применение ER-диаграмм

Модели «сущность-связь» традиционно используют для разработки программного обеспечения. При этом для метода нет конкретной области разработки: для создания любого ПО нужно работать с данными и транслировать их пользователям. Поэтому ER-модели строят и для интернет-магазина, и для корпоративного портала компании.

Обычно ER-модель создают в двух случаях:

- когда перед началом проекта ещё не понятно, с какими данными предстоит работать;
- когда нужно создать новую базу данных или добавить таблицу в уже существующую.

Чем больше в системе сущностей и связей, тем важнее построить ER-модель до начала разработки ПО.

На практике над простыми системами можно работать без концептуальной ER-модели. Например, программа для выдачи талонов электронной очереди — простая система, в которой всего две сущности — номер окна и номер очереди.

Символы и нотации ER-диаграмм

ER-модель — это общее представление данных, ER-диаграмма — представление модели, а нотация — графический язык для представления модели.

Объясним на примере анатомии человека. Устройство человеческого организма — это модель. Её можно описать текстом, изобразить на картинке, перечислить все органы в таблице. Всё это разные представления одной и той же модели. Символы, с помощью которых описывают модель, — это нотации.

Для того чтобы построить ER-диаграмму, можно использовать разные нотации. Три самые известные из них:

1. Нотация IDEF1X. Её относят к фундаментальным, но на практике давно не используют, потому что есть более удобные варианты.

2. Нотация Чена. Классическая нотация, которая состоит из простых символов — прямоугольников, овалов и линий. Из-за этого нотацию часто используют для концептуальных моделей, которые презентуют заказчику. Человеку, который далёк от аналитики данных, проще разобраться в понятных диаграммах со знакомыми символами.

3. Нотация Мартина. Её ещё называют «воронья лапка» (от англ. Crow's Foot). Она компактнее нотации Чена, поэтому её используют для построения ER-моделей логического уровня, когда нужно описать в модели все атрибуты сущностей.

В нотациях Чена и Мартина есть одинаковые элементы: сущности, атрибуты и связи. Но эти элементы диаграмм обозначают разными символами.

Нотация Мартина

Сущность изображается прямоугольником, внутри которого указано ее имя жирным шрифтом и список ее атрибутов (идентифицирующий атрибут подчеркнут/выделен жирным), а связь – линией, название которой располагается над ней и ее вид в месте соединения с сущностью определяет кардинальность связи (рис. 1)

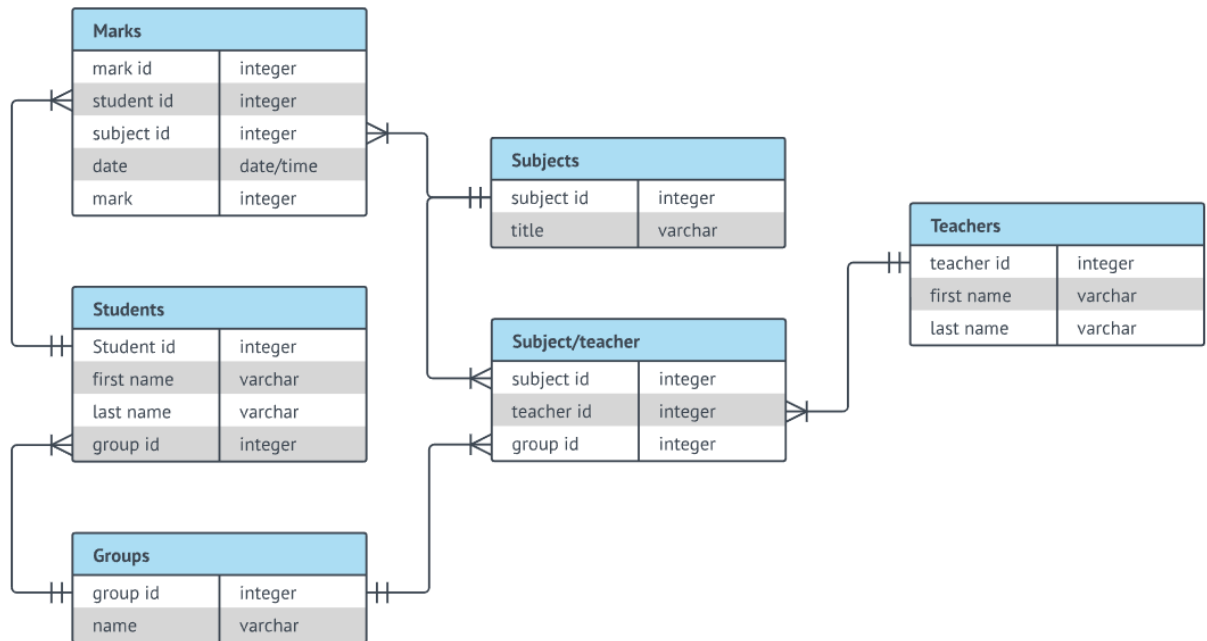
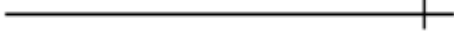
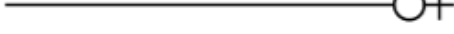


Рис. 1 Пример нотации Мартина

В нотации Мартина для обозначения связей между сущностями используются следующие элементы:

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many

Как создать простую ER-диаграмму

Вот пошаговый алгоритм для создания простой ER-диаграммы «сущность-связь»:

1. Определить сущности. Чтобы собрать все сущности будущего проекта, системные аналитики общаются с заказчиком и будущими пользователями ПО: сотрудниками или клиентами компании. Например, если нужно разработать ПО для ветеринарной клиники, системный аналитик проведёт интервью с руководителем клиники, сотрудниками, врачами и клиентами, которые будут записываться на приём. На этом этапе обычно создают концептуальную модель и согласовывают её с заказчиком.
2. Определить атрибуты. Системный аналитик детализирует информацию, собранную во время интервью, и описывает характеристики сущностей. Если данных не хватает, нужно повторно опросить заинтересованных лиц.
3. Определить связи между сущностями. На этом этапе выясняют, какие сущности связаны между собой. Например, пациенты и медицинская карта, филиал клиники и врачи, которые ведут приём.

4. Определить типы и характеристики связей. Например, пациенты и медицинская карта — это связь «один-к-одному», врач и день приёма — «один-ко-многим».

Затем ищут идентифицирующие связи между сущностями и определяют, какая из сущностей родительская. Допустим, у клиники есть филиалы — А, В и С. В каждом филиале есть кабинеты под номерами от 1 до 5. Это значит, что нельзя использовать номер кабинета без уточнения, в каком филиале он находится. Филиал — родительская сущность, а связь между филиалом и кабинетом — идентифицирующая.

5. Проверить ER-модель. После завершения работы над ER-моделью системный аналитик проверяет, нет ли в ней лишних сущностей, дубликатов данных и косвенных связей между данными в одной таблице. Такую проверку называют нормализацией данных.

1.3 Язык SQL

SQL (аббр. от англ. Structured Query Language — «язык структурированных запросов») — декларативный язык программирования, применяемый для создания, модификации и управления данными в реляционной базе данных, управляемой соответствующей системой управления базами данных.

Является, прежде всего, информационно-логическим языком, предназначенным для описания, изменения и извлечения данных, хранимых в реляционных базах данных. В общем случае SQL (без ряда современных расширений) считается языком программирования неполным по Тьюрингу, но вместе с тем стандарт языка спецификацией SQL/PSM предусматривает возможность его процедурных расширений.

Изначально SQL был основным способом работы пользователя с базой данных и позволял выполнять следующий набор операций:

- создание в базе данных новой таблицы;
- добавление в таблицу новых записей;
- изменение записей;
- удаление записей;
- выборка записей из одной или нескольких таблиц (в соответствии с заданным условием);
- изменение структур таблиц.

Элементы

Язык SQL представляет собой совокупность операторов, инструкций, вычисляемых функций.

Согласно общепринятому стилю программирования, операторы (и другие зарезервированные слова) в SQL обычно рекомендуется писать прописными буквами.

Операторы SQL делятся на:

операторы определения данных (Data Definition Language, DDL):

- CREATE создаёт объект базы данных (саму базу, таблицу, представление, пользователя и так далее),
- ALTER изменяет объект,
- DROP удаляет объект;

операторы манипуляции данными (Data Manipulation Language, DML):

- SELECT выбирает данные, удовлетворяющие заданным условиям,
- INSERT добавляет новые данные,
- UPDATE изменяет существующие данные,
- DELETE удаляет данные;

операторы определения доступа к данным (Data Control Language, DCL):

- GRANT предоставляет пользователю (группе) разрешения на определённые операции с объектом,
- REVOKE отзывает ранее выданные разрешения,
- DENY задаёт запрет, имеющий приоритет над разрешением;

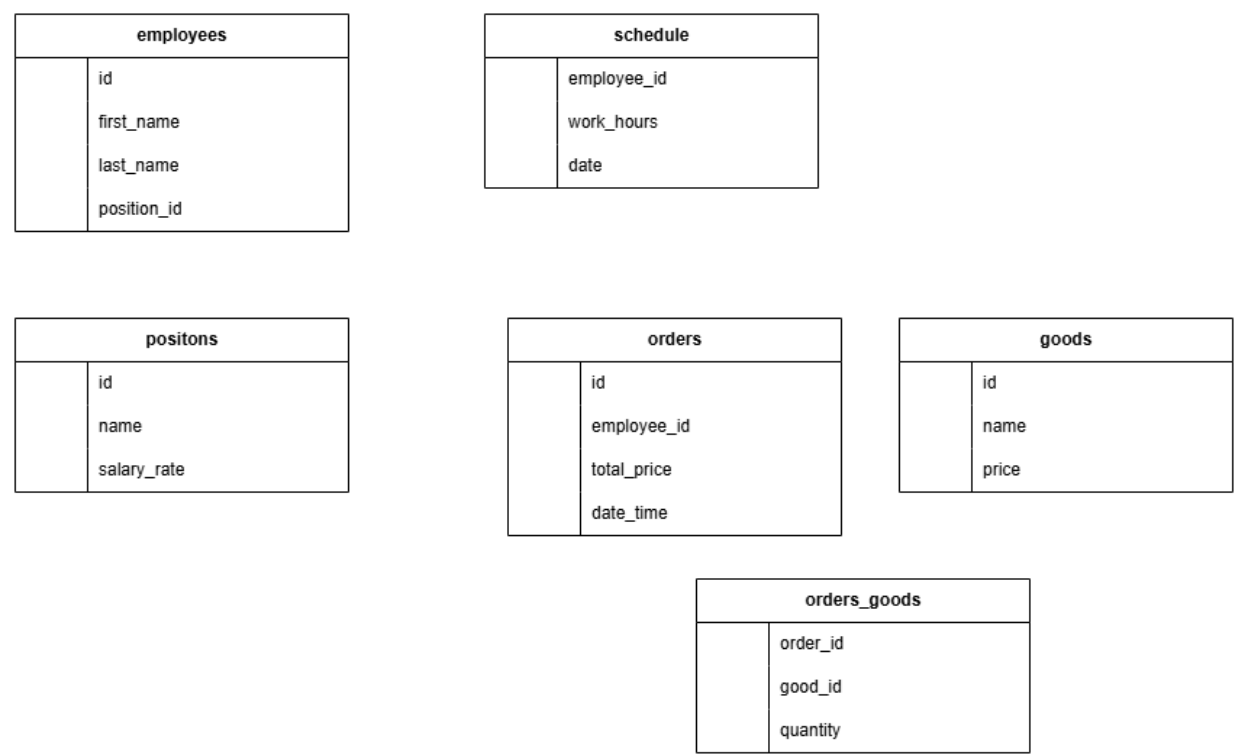
операторы управления транзакциями (Transaction Control Language, TCL):

- COMMIT применяет транзакцию,
- ROLLBACK откатывает все изменения, сделанные в контексте текущей транзакции,
- SAVEPOINT делит транзакцию на более мелкие участки.

2. Практические задания

Часть 1

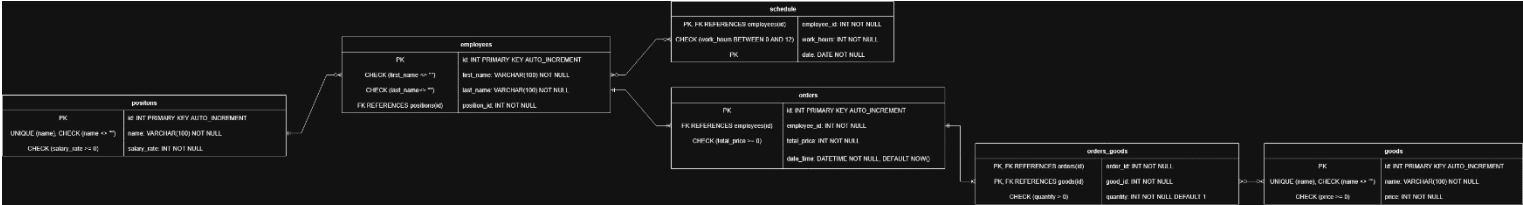
2.1 Дополнить ER-диаграмму в нотации Мартина для следующей базы данных:



Получите исходный файл drawio с указанной БД. Дополните диаграмму типами атрибутов, ограничениями (restrictions), а также добавьте связи между таблицами.

Результат:

В хорошем качестве



2.2 Создание базы данных

Для работы подключитесь к удаленному серверу. Адрес сервера, данные учетной записи узнать у преподавателя.

Для подключения к серверу используйте приложение MobaXTerm

Команда для подключения к серверу:

```
ssh user@host
```

При выполнении команды будет запрошен пароль. Введите его.

После входа на сервер необходимо подключиться к серверу MySQL:

```
mysql -u rudn_student_2 -p
```

данная команда после выполнения запросит пароль для входа, введите его.

После подключения к mysql первым шагом необходимо создать базу данных. Для создания базы данных выполните команду:

```
CREATE DATABASE database_name DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;  
где database_name – название базы данных, которое составляется по шаблону  
fast_food_resto_номерстудбилета
```

Результатом выполнения команды должна быть созданная база данных. Выберите ее для дальнейшей работы:

```
USE fast_food_resto_номерстудбилета;
```

Решение/команды:

```
PS C:\Users\hatar> ssh rudn_student@5.188.30.207
rudn_student@5.188.30.207's password:
Last failed login: Sat Sep 20 12:31:33 MSK 2025 from 80.250.174.190 on ssh:notty
There were 3 failed login attempts since the last successful login.
Last login: Sat Sep 20 12:31:10 2025 from public-nat-14.vpngate.v4.open.ad.jp
[rudn_student@s7aabbed2 ~]$ ls
-server-key.pem      1032244655  1032240586  1132242461  1132243698  1132247425  USE
1031244929@pfur.ru  1032244892  1132240588  1132242962  1132243699  1132247426  USEfast_food_resto_1032242391
1032231391          1032244918  1132241486  1132243256  1132243708  1132247427  check
1032240489          1032244929  1132242343  1132243257  1132246185  1132247428  lab.certs
1032240722          1032245465  1132242344  1132243258  1132246665  1132249952  root.pem
1032242353          1032248200  1132242345  1132243259  1132246666  N            rootCA.key
1032242404          1032249579  1132242346  1132243699  1132246667  ON           ??
1032240652          1032249820  1132242347  1132243690  1132246668  SHOW
1032240654          1032240585  1132242348  1132243697  1132246671  TABLE positions;

[rudn_student@s7aabbed2 ~]$ mysql -u rudn_student_2 -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 239618
Server version: 8.0.30 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> help

mysql> CREATE DATABASE fast_food_resto_1032259220 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
Query OK, 1 row affected (0.00 sec)

mysql> |
```

```
mysql> USE fast_food_resto_1032259220
Database changed
mysql> |
```

2.3 Создание таблиц в базе данных

После создания базы данных необходимо заполнить ее таблицами. Используя заполненную диаграмму, создайте все таблицы.

Пример команды для создания таблицы:

```
CREATE TABLE IF NOT EXISTS `chief` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `first_name` varchar(255) NOT NULL,  
  `second_name` varchar(255) NOT NULL,  
  `division_id` int DEFAULT NULL,  
  `email` varchar(255) DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Решение/команды:

```
CREATE TABLE IF NOT EXISTS `goods` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `name` varchar(100) NOT NULL,  
  `price` int NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE IF NOT EXISTS `positions` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `name` varchar(100) NOT NULL,  
  `salary_rate` int NOT NULL,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `chk_name_not_empty` CHECK (`name` <> ''),  
  CONSTRAINT `chk_positive_salary` CHECK (`salary_rate` > 0)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE IF NOT EXISTS `employees` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `first_name` varchar(100) NOT NULL,  
  `last_name` varchar(100) NOT NULL,  
  `position_id` int NOT NULL,  
  PRIMARY KEY (`id`),  
  FOREIGN KEY (`position_id`) REFERENCES `positions` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE IF NOT EXISTS `schedule` (  
  `employee_id` int NOT NULL,  
  `work_hours` int NOT NULL,  
  `date` date NOT NULL,  
  PRIMARY KEY (`employee_id`, `date`),  
  FOREIGN KEY (`employee_id`) REFERENCES `employees` (`id`),  
  CONSTRAINT `chk_work_time_limits` CHECK (`work_hours` BETWEEN 0 AND 12)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE IF NOT EXISTS `orders` (  
  `id` int PRIMARY KEY AUTO_INCREMENT,  
  `employee_id` int NOT NULL,  
  `total_price` int NOT NULL,  
  `date_time` datetime NOT NULL DEFAULT NOW(),  
  CONSTRAINT `fk_orders_employee` FOREIGN KEY (`employee_id`) REFERENCES  
  `employees` (`id`),  
  CONSTRAINT `chk_total_price` CHECK (`total_price` >= 0)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE IF NOT EXISTS `orders_goods` (  
  `order_id` int NOT NULL,  
  `good_id` int NOT NULL,  
  `quantity` int NOT NULL DEFAULT 1,  
  PRIMARY KEY (`order_id`, `good_id`),  
  FOREIGN KEY (`order_id`) REFERENCES `orders`(`id`),  
  FOREIGN KEY (`good_id`) REFERENCES `goods`(`id`),  
  CONSTRAINT `chk_quantity_positive` CHECK (`quantity` > 0)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

2.4 Создание restrictions на внешние ключи

Для сохранения целостности данных в базе таблицы могут быть связаны друг с другом через внешний ключ. На данную связь можно наложить ограничения, которые позволят сохранять данные в базе в консистентном состоянии.

Для таблиц с внешними ключами добавьте ограничения на внешние ключи путем использования команды ALTER TABLE.

Пример такой команды:

```
ALTER TABLE `employees` ADD CONSTRAINT `fk-employee-position` FOREIGN KEY (`position_id`) REFERENCES `positions` (`id`) ON DELETE SET NULL ON UPDATE CASCADE;
```

Количество ограничений на ключи должно быть равно количеству связей между таблицами.

employees – schedule (on delete restrict on update cascade)

employees – orders (on delete restrict on update cascade)

employees – positions (on delete restrict on update cascade)

orders – orders_goods (on delete cascade on update cascade)

orders_goods – goods (on delete restrict on update cascade)

Решение/команды:

```
mysql> ALTER TABLE `employees` DROP FOREIGN KEY `employees_ibfk_1`, ADD CONSTRAINT `employees_ibfk_1` FOREIGN KEY (`position_id`) REFERENCES `positions` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;
ERROR 1826 (HY000): Duplicate foreign key constraint name 'employees_ibfk_1'
mysql> ALTER TABLE `employees` DROP FOREIGN KEY `employees_ibfk_1`;
Query OK, 0 rows affected (1.75 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `employees` ADD CONSTRAINT `employees_ibfk_1` FOREIGN KEY (`position_id`) REFERENCES `positions` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> ALTER TABLE `schedule` DROP FOREIGN KEY `schedule_ibfk_1`;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `schedule` ADD CONSTRAINT `schedule_ibfk_1` FOREIGN KEY (`employee_id`) REFERENCES `employees` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> ALTER TABLE `orders` DROP FOREIGN KEY `fk_orders_employee`;
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `orders` ADD CONSTRAINT `fk_orders_employee` FOREIGN KEY (`employee_id`) REFERENCES `employees` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> ALTER TABLE `orders_goods` DROP FOREIGN KEY `orders_goods_ibfk_1`;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `orders_goods` ADD CONSTRAINT `orders_goods_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> ALTER TABLE `orders_goods` DROP FOREIGN KEY `orders_goods_ibfk_2`;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `orders_goods` ADD CONSTRAINT `orders_goods_ibfk_2` FOREIGN KEY (`good_id`) REFERENCES `goods` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Часть 2.

2.5 Проверка на наличие ограничений (constraints) в таблице

С помощью команды `SHOW CREATE TABLE `имя_таблицы`` можно посмотреть структуру таблицы и ее ограничения. Выполнить эту команду для всех таблиц в базе данных.

Решение/команды:

```
mysql> SHOW CREATE TABLE goods;
+-----+-----+
| Table | Create Table
+-----+-----+
| goods | CREATE TABLE `goods` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `price` int NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

```
mysql> SHOW CREATE TABLE positions;
+-----+-----+
| Table | Create Table
+-----+-----+
| positions | CREATE TABLE `positions` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `salary_rate` int NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `chk_name_not_empty` CHECK ((`name` <> _utf8mb4''))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SHOW CREATE TABLE employees;
+-----+-----+
| Table | Create Table
+-----+-----+
| employees | CREATE TABLE `employees` (
  `id` int NOT NULL AUTO_INCREMENT,
  `first_name` varchar(100) NOT NULL,
  `last_name` varchar(100) NOT NULL,
  `position_id` int NOT NULL,
  PRIMARY KEY (`id`),
  KEY `fk_employee_position` (`position_id`),
  CONSTRAINT `fk_employee_position` FOREIGN KEY (`position_id`) REFERENCES `positions` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `chk_first_name_no_digits` CHECK ((not(regexp_like('first_name',_utf8mb4'[0-9]')))),
  CONSTRAINT `chk_first_name_not_empty` CHECK ((char_length(trim('first_name')) > 0)),
  CONSTRAINT `chk_last_name_no_digits` CHECK ((not(regexp_like('last_name',_utf8mb4'[0-9]')))),
  CONSTRAINT `chk_last_name_not_empty` CHECK ((char_length(trim('last_name')) > 0))
) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SHOW CREATE TABLE orders;
+-----+
| Table | Create Table
+-----+
| orders | CREATE TABLE `orders` (
  `id` int NOT NULL AUTO_INCREMENT,
  `employee_id` int NOT NULL,
  `total_price` int NOT NULL,
  `date_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `fk_orders_employee` (`employee_id`),
  CONSTRAINT `fk_orders_employee` FOREIGN KEY (`employee_id`) REFERENCES `employees` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `chk_total_price` CHECK ((`total_price` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SHOW CREATE TABLE orders_goods;
+-----+
| Table | Create Table
+-----+
| orders_goods | CREATE TABLE `orders_goods` (
  `order_id` int NOT NULL,
  `good_id` int NOT NULL,
  `quantity` int NOT NULL DEFAULT '1',
  PRIMARY KEY (`order_id`, `good_id`),
  KEY `orders_goods_ibfk_2` (`good_id`),
  CONSTRAINT `orders_goods_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `orders_goods_ibfk_2` FOREIGN KEY (`good_id`) REFERENCES `goods` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `chk_quantity_positive` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+
1 row in set (0.01 sec)
```

2.6 Вставка записей в таблицу, нарушающих ограничения

На таблице `goods` выполнить команды вставки значений (INSERT INTO ...), которые нарушают установленные ограничения на таблицу. Т.о. все операции должны быть завершены неудачно

- price < 0

```
mysql> ALTER TABLE goods ADD CONSTRAINT chk_price_positive CHECK (price > 0);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> INSERT INTO goods (name, price) VALUES ('Pizza', -100);
ERROR 3819 (HY000): Check constraint 'chk_price_positive' is violated.
```

- Не передан price

```
mysql> INSERT INTO goods (name) VALUES ('Pizza2');  
ERROR 1364 (HY000): Field 'price' doesn't have a default value  
mysql>
```

- name не уникален

```
mysql> ALTER TABLE goods ADD CONSTRAINT uk_goods_name UNIQUE (name);  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO goods (name, price) VALUES ('Pizza', 200);  
Query OK, 1 row affected (0.00 sec)  
  
mysql> INSERT INTO goods (name, price) VALUES ('Pizza', 100);  
ERROR 1062 (23000): Duplicate entry 'Pizza' for key 'goods.uk_goods_name'  
mysql>
```

- name – пустая строка ""

```
mysql> ALTER TABLE goods ADD CONSTRAINT chk_good_name_not_empty CHECK (name <> '');  
Query OK, 0 rows affected (0.02 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO goods (name, price) VALUES ('', 100);  
ERROR 3819 (HY000): Check constraint 'chk_good_name_not_empty' is violated.  
mysql> |
```

2.7 Проверка работы ограничений на внешние ключи

Внимание! Перед выполнением данного задания необходимо заполнить базу данных. Попросите преподавателя выполнить данную процедуру.

При наличии ограничений на внешние ключи операции удаления и обновления в связанных таблицах могут быть произведены каскадно или же запрещены.

- На примере таблицы employees и schedule – попробовать удалить сотрудника, у которого есть расписание. Операция должна завершиться неудачей

```
rudn_student@s7aabb22:~  
mysql> DELETE FROM employees WHERE id = 1;  
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails ('fast_food_resto_1032259220`.`orders', CONSTRAINT 'fk_orders_employee' FOREIGN KEY ('employee_id') REFERENCES 'employees' ('id') ON DELETE RESTRICT ON UPDATE CASCADE)  
mysql> SELECT * FROM employees WHERE id = 1;  
+----+-----+-----+-----+  
| id | first_name | last_name | position_id |  
+----+-----+-----+-----+  
| 1 | Daniel | Wright | 1 |  
+----+-----+-----+-----+  
1 row in set (0.00 sec)  
  
mysql>
```

- На примере таблиц orders и orders_goods: попробовать удалить запись из таблицы orders с любым id. Проверить, что в таблице orders_goods каскадно удалились записи, у которых был order_id = id

```
rudn_student@s7aabbed2:~  
mysql> SELECT * FROM orders WHERE id = 1;  
+-----+-----+-----+-----+  
| id | employee_id | total_price | date_time |  
+-----+-----+-----+-----+  
| 1 | 1 | 100 | 2024-08-12 17:00:00 |  
+-----+-----+-----+-----+  
1 row in set (0.01 sec)  
  
mysql> SELECT * FROM orders_goods WHERE order_id = 1;  
+-----+-----+-----+  
| order_id | good_id | quantity |  
+-----+-----+-----+  
| 1 | 6 | 3 |  
| 1 | 10 | 2 |  
| 1 | 11 | 1 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> DELETE FROM orders WHERE id = 1;  
Query OK, 1 row affected (0.00 sec)  
  
mysql> SELECT * FROM orders_goods WHERE order_id = 1;  
Empty set (0.00 sec)  
  
mysql>
```

Часть 3. Работа с данными в базе

Выполнить команды, которые выполняют указанные ниже условия

Извлечение данных:

- Без указания столбцов (все столбцы)
- С указанием конкретных столбцов
- С выводом уникальных значений (DISTINCT)
- С ограничением количества выводимых записей (LIMIT)

```
mysql> SELECT * FROM goods;
+----+-----+-----+
| id | name                | price |
+----+-----+-----+
| 1  | Small Cola          | 40    |
| 2  | Medium Cola         | 55    |
| 3  | Large Cola          | 70    |
| 4  | Hamburger           | 50    |
| 5  | Cheeseburger         | 65    |
| 6  | Double Cheeseburger | 105   |
| 7  | Big mac              | 200   |
| 8  | Hot toast            | 80    |
| 9  | Fish burger          | 110   |
| 10 | Small fries          | 40    |
| 11 | Medium fries         | 70    |
| 12 | Large fries          | 90    |
| 13 | Sweet and sour sauce | 15    |
| 14 | Barbecue sauce       | 15    |
| 15 | Cheese sauce         | 15    |
| 16 | Nuggets 6            | 150   |
| 17 | Nuggets 9            | 200   |
| 18 | Nuggets 12           | 220   |
| 19 | Caesar roll          | 200   |
+----+-----+-----+
19 rows in set (0.01 sec)

mysql>
```

```
mysql> SELECT id, name FROM goods;
+----+-----+
| id | name                |
+----+-----+
| 14 | Barbecue sauce       |
| 7  | Big mac              |
| 19 | Caesar roll          |
| 15 | Cheese sauce         |
| 5  | Cheeseburger         |
| 6  | Double Cheeseburger |
| 9  | Fish burger          |
| 4  | Hamburger           |
| 8  | Hot toast            |
| 3  | Large Cola          |
| 12 | Large fries          |
| 2  | Medium Cola         |
| 11 | Medium fries         |
| 18 | Nuggets 12           |
| 16 | Nuggets 6            |
| 17 | Nuggets 9            |
| 1  | Small Cola          |
| 10 | Small fries          |
| 13 | Sweet and sour sauce |
+----+-----+
19 rows in set (0.00 sec)

mysql>
```

```
rudn_student@s7aabb2:~  
mysql> SELECT DISTINCT price FROM goods;  
+-----+  
| price |  
+-----+  
| 40 |  
| 55 |  
| 70 |  
| 50 |  
| 65 |  
| 105 |  
| 200 |  
| 80 |  
| 110 |  
| 90 |  
| 15 |  
| 150 |  
| 220 |  
+-----+  
13 rows in set (0.00 sec)  
  
mysql>
```

```
rudn_student@s7aabb2:~  
mysql> SELECT * FROM goods  
-> LIMIT 5;  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 1 | Small Cola | 40 |  
| 2 | Medium Cola | 55 |  
| 3 | Large Cola | 70 |  
| 4 | Hamburger | 50 |  
| 5 | Cheeseburger | 65 |  
+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql>
```

Сортировка (использовать в командах DESC/ASC):

- Обычная сортировка по столбцу (ORDER BY)
- Сортировка по нескольким столбцам

```
rudn_student@s7aabb2:~  
mysql> SELECT * FROM goods  
-> ORDER BY price ASC;  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 15 | Cheese sauce | 15 |  
| 14 | Barbecue sauce | 15 |  
| 13 | Sweet and sour sauce | 15 |  
| 1 | Small Cola | 40 |  
| 10 | Small fries | 40 |  
| 4 | Hamburger | 50 |  
| 2 | Medium Cola | 55 |  
| 5 | Cheeseburger | 65 |  
| 11 | Medium fries | 70 |  
| 3 | Large Cola | 70 |  
| 8 | Hot toast | 80 |  
| 12 | Large fries | 90 |  
| 6 | Double Cheeseburger | 105 |  
| 9 | Fish burger | 110 |  
| 16 | Nuggets 6 | 150 |  
| 7 | Big mac | 200 |  
| 17 | Nuggets 9 | 200 |  
| 19 | Caesar roll | 200 |  
| 18 | Nuggets 12 | 220 |  
+-----+-----+-----+  
19 rows in set (0.00 sec)  
  
mysql>
```

```

mysql> SELECT * FROM goods
-> ORDER BY price ASC, name DESC;
+-----+
| id | name           | price |
+-----+
| 13 | Sweet and sour sauce | 15    |
| 15 | Cheese sauce    | 15    |
| 14 | Barbecue sauce  | 15    |
| 10 | Small fries     | 40    |
| 1  | Small Cola      | 40    |
| 4  | Hamburger       | 50    |
| 2  | Medium Cola     | 55    |
| 5  | Cheeseburger    | 65    |
| 11 | Medium fries    | 70    |
| 3  | Large Cola      | 70    |
| 8  | Hot toast       | 80    |
| 12 | Large fries     | 90    |
| 6  | Double Cheeseburger | 105   |
| 9  | Fish burger     | 110   |
| 16 | Nuggets 6       | 150   |
| 17 | Nuggets 9       | 200   |
| 19 | Caesar roll     | 200   |
| 7  | Big mac         | 200   |
| 18 | Nuggets 12      | 220   |
+-----+
19 rows in set (0.01 sec)

```

Фильтрация выводимых данных:

- Несколько условий в WHERE (использовать AND/OR)
- Фильтрация по конкретному значению столбца
- Проверка на неравенство
- С заданием диапазона, в котором должны быть значения
- С использованием оператора IN / NOT IN
- С использованием оператора LIKE. Использовать метасимволы %, _, []

```

mysql> SELECT * FROM goods
-> WHERE price > 50 AND name LIKE '%Cola%';
+-----+
| id | name           | price |
+-----+
| 2  | Medium Cola    | 55    |
| 3  | Large Cola     | 70    |
+-----+
2 rows in set (0.00 sec)

mysql>

```

```

mysql> SELECT * FROM goods
-> WHERE price = 40;
+-----+
| id | name           | price |
+-----+
| 1  | Small Cola     | 40    |
| 10 | Small fries    | 40    |
+-----+
2 rows in set (0.00 sec)

mysql>

```

```
rudn_student@s7aabbed2:~  
mysql> SELECT * FROM goods  
-> WHERE price <> 40;  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 2 | Medium Cola | 55 |  
| 3 | Large Cola | 70 |  
| 4 | Hamburger | 50 |  
| 5 | Cheeseburger | 65 |  
| 6 | Double Cheeseburger | 105 |  
| 7 | Big mac | 200 |  
| 8 | Hot toast | 80 |  
| 9 | Fish burger | 110 |  
| 11 | Medium fries | 70 |  
| 12 | Large fries | 90 |  
| 13 | Sweet and sour sauce | 15 |  
| 14 | Barbecue sauce | 15 |  
| 15 | Cheese sauce | 15 |  
| 16 | Nuggets 6 | 150 |  
| 17 | Nuggets 9 | 200 |  
| 18 | Nuggets 12 | 220 |  
| 19 | Caesar roll | 200 |  
+-----+-----+-----+  
17 rows in set (0.00 sec)  
mysql>
```

```
rudn_student@s7aabbed2:~  
mysql> SELECT * FROM goods  
-> WHERE price BETWEEN 50 AND 100;  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 2 | Medium Cola | 55 |  
| 3 | Large Cola | 70 |  
| 4 | Hamburger | 50 |  
| 5 | Cheeseburger | 65 |  
| 8 | Hot toast | 80 |  
| 11 | Medium fries | 70 |  
| 12 | Large fries | 90 |  
+-----+-----+-----+  
7 rows in set (0.00 sec)  
mysql>
```

```
rudn_student@s7aabbed2:~  
mysql> SELECT * FROM goods  
-> WHERE price IN (40, 70, 200);  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 1 | Small Cola | 40 |  
| 3 | Large Cola | 70 |  
| 7 | Big mac | 200 |  
| 10 | Small fries | 40 |  
| 11 | Medium fries | 70 |  
| 17 | Nuggets 9 | 200 |  
| 19 | Caesar roll | 200 |  
+-----+-----+-----+  
7 rows in set (0.00 sec)  
mysql> SELECT * FROM goods  
-> WHERE price NOT IN (40, 70, 200);  
+-----+-----+-----+  
| id | name | price |  
+-----+-----+-----+  
| 2 | Medium Cola | 55 |  
| 4 | Hamburger | 50 |  
| 5 | Cheeseburger | 65 |  
| 6 | Double Cheeseburger | 105 |  
| 8 | Hot toast | 80 |  
| 9 | Fish burger | 110 |  
| 12 | Large fries | 90 |  
| 13 | Sweet and sour sauce | 15 |  
| 14 | Barbecue sauce | 15 |  
| 15 | Cheese sauce | 15 |  
| 16 | Nuggets 6 | 150 |  
| 18 | Nuggets 12 | 220 |  
+-----+-----+-----+  
12 rows in set (0.00 sec)  
mysql>
```

```
mysql> SELECT * FROM goods WHERE name LIKE 'Nuggets _';
+----+-----+-----+
| id | name      | price |
+----+-----+-----+
| 16 | Nuggets 6 | 150   |
| 17 | Nuggets 9 | 200   |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM goods
-> WHERE name LIKE 'S%';
+----+-----+-----+
| id | name              | price |
+----+-----+-----+
| 1  | Small Cola        | 40    |
| 10 | Small fries       | 40    |
| 13 | Sweet and sour sauce | 15    |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM goods
-> WHERE name REGEXP '^[B-C]';
+----+-----+-----+
| id | name              | price |
+----+-----+-----+
| 5  | Cheeseburger      | 65    |
| 7  | Big mac           | 200   |
| 14 | Barbecue sauce    | 15    |
| 15 | Cheese sauce      | 15    |
| 19 | Caesar roll       | 200   |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

Часть 4. Объединение таблиц. Агрегирующие функции. Группирование с фильтрацией

Задания:

- Для записей таблице orders вставить посчитанные значения столбца total_price исходя из данных заказа. Данные заказа брать из смежных таблиц orders_goods, goods. В задании используются подзапросы, update, join, sum.

```
mysql> UPDATE orders o
-> JOIN (
->   SELECT og.order_id, SUM(g.price * og.quantity) AS total_sum
->   FROM orders_goods og
->   JOIN goods g ON og.good_id = g.id
->   GROUP BY og.order_id
-> ) AS totals ON o.id = totals.order_id
-> SET o.total_price = totals.total_sum;
Query OK, 3964 rows affected (0.25 sec)
Rows matched: 3975  Changed: 3964  Warnings: 0

mysql> SELECT * orders;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
for the right syntax to use near 'orders' at line 1
mysql> SELECT * FROM orders;
+----+-----+-----+-----+
| id | employee_id | total_price | date_time |
+----+-----+-----+-----+
| 2  | 1           | 900        | 2024-11-29 14:03:00 |
| 3  | 1           | 695        | 2024-11-29 02:42:00 |
| 4  | 1           | 430        | 2024-11-29 17:10:00 |
| 5  | 1           | 385        | 2024-11-29 08:55:00 |
+----+-----+-----+-----+
```

- Определить, какой товар не покупали ни разу за все время? Вывести его название

```

mysql> SELECT g.name
-> FROM goods g
-> LEFT JOIN orders_goods og ON g.id = og.good_id
-> WHERE og.good_id IS NULL;
+-----+
| name |
+-----+
| Small Cola |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM goods WHERE name="Small Cola";
+----+-----+-----+
| id | name | price |
+----+-----+-----+
| 1 | Small Cola | 40 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM orders_goods WHERE good_id=1;
Empty set (0.00 sec)

mysql>

```

- Какой сотрудник выполнил больше всех заказов? Вывести фамилию, имя, количество заказов

```

mysql> SELECT e.last_name, e.first_name, COUNT(o.id) AS order_count
-> FROM employees e
-> JOIN orders o ON e.id = o.employee_id
-> GROUP BY e.id
-> ORDER BY order_count DESC
-> LIMIT 1;
+-----+-----+-----+
| last_name | first_name | order_count |
+-----+-----+-----+
| Rogers | Kathleen | 423 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

- Вывести среднюю сумму заказа за все время

```

mysql> SELECT AVG(total_price) AS avg_order_sum
-> FROM orders;
+-----+
| avg_order_sum |
+-----+
| 542.4679 |
+-----+
1 row in set (0.00 sec)

mysql>

```

- Вывести среднюю сумму заказа за все время с разбивкой по месяцам

```
mysql> SELECT
-> AS avg_order_sum
FROM orders
GROUP BY DATE_FORMAT(date_time, '%Y-%m')
ORDER BY order_month;
DATE_FORMAT(date_time, '%Y-%m') AS order_month,
-> AVG(total_price) AS avg_order_sum
-> FROM orders
-> GROUP BY DATE_FORMAT(date_time, '%Y-%m')
-> ORDER BY order_month;
+-----+-----+
| order_month | avg_order_sum |
+-----+-----+
| 2024-01     | 553.9370      |
| 2024-02     | 566.3916      |
| 2024-03     | 561.2297      |
| 2024-04     | 520.4437      |
| 2024-05     | 553.1381      |
| 2024-06     | 522.0245      |
| 2024-07     | 515.7036      |
| 2024-08     | 541.6172      |
| 2024-09     | 559.5277      |
| 2024-10     | 532.0031      |
| 2024-11     | 531.6505      |
| 2024-12     | 550.5212      |
+-----+-----+
12 rows in set (0.01 sec)

mysql>
```

- Выбрать любой товар из таблицы goods (за исключением товара из задания 2). Вывести рейтинг сотрудников по количеству проданного товара, причем выводиться должны сотрудники, которые продали больше среднего по всем сотрудникам (подзапросы, ORDER, GROUP BY, HAVING)

```
mysql> SELECT e.last_name, e.first_name, SUM(og.quantity) AS total_sold
-> FROM employees e
-> _id = 2
GROUP BY JOIN orders o ON e.id = o.employee_id
-> JOIN orders_goods og ON o.id = og.order_id
-> WHERE og.good_id = 2
-> GROUP BY e.id
-> HAVING total_sold > (
-> SELECT AVG(s.total_sold)
-> CT e2.id, SUM(og FROM (
-> SELECT e2.id, SUM(og2.quantity) AS total_sold
-> FROM employees e2
-> JOIN orders o2 ON e2.id = o2.employee_id
-> d_id = 2
-> JOIN orders_goods og2 ON o2.id = og2.order_id
-> WHERE og2.good_id = 2
-> GROUP BY e2.id
-> ) AS s
-> )
-> ORDER BY total_sold DESC;
+-----+-----+-----+
| last_name | first_name | total_sold |
+-----+-----+-----+
| Rice      | Eric       | 149        |
| Weaver    | Janet      | 149        |
| Owens     | Carol      | 135        |
| Torres    | Helen      | 123        |
+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

Часть 5. Работа с транзакциями

Задания

- Вывести переменные окружения, которые отвечают за настройки транзакций.

SHOW VARIABLES LIKE “%transaction%”

```
mysql> SHOW VARIABLES LIKE '%transaction%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_direct_non_transactional_updates | OFF |
| binlog_transaction_compression | OFF |
| binlog_transaction_compression_level_zstd | 3 |
| binlog_transaction_dependency_history_size | 25000 |
| binlog_transaction_dependency_tracking | COMMIT_ORDER |
| performance_schema_events_transactions_history_long_size | 10000 |
| performance_schema_events_transactions_history_size | 10 |
| replica_transaction_retries | 10 |
| session_track_transaction_info | OFF |
| slave_transaction_retries | 10 |
| transaction_alloc_block_size | 8192 |
| transaction_allow_batching | OFF |
| transaction_isolation | REPEATABLE-READ |
| transaction_prealloc_size | 4096 |
| transaction_read_only | OFF |
| transaction_write_set_extraction | XXHASH64 |
+-----+-----+
16 rows in set (0.01 sec)

mysql> |
```

- Попереклывать уровень изоляции транзакций на глобальном и сессионном уровне. Проверить, изменение уровня изоляции можно через переменные окружения

SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL ____;

```
mysql> SELECT @@GLOBAL.transaction_isolation, @@SESSION.transaction_isolation;
+-----+-----+
| @@GLOBAL.transaction_isolation | @@SESSION.transaction_isolation |
+-----+-----+
| REPEATABLE-READ | REPEATABLE-READ |
+-----+-----+
1 row in set (0.00 sec)

mysql> SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
ERROR 1227 (42000): Access denied; you need (at least one of) the SUPER or SYSTEM_VARIABLES_ADMIN privilege(s) for this operation
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @@GLOBAL.transaction_isolation, @@SESSION.transaction_isolation;
+-----+-----+
| @@GLOBAL.transaction_isolation | @@SESSION.transaction_isolation |
+-----+-----+
| REPEATABLE-READ | READ-UNCOMMITTED |
+-----+-----+
1 row in set (0.00 sec)

mysql> SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @@GLOBAL.transaction_isolation, @@SESSION.transaction_isolation;
+-----+-----+
| @@GLOBAL.transaction_isolation | @@SESSION.transaction_isolation |
+-----+-----+
| REPEATABLE-READ | REPEATABLE-READ |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

- Написать команду для выполнения транзакции на таблицах orders и orders_goods. В рамках транзакции создать новый заказ и привязать к

нему несколько товаров. После завершения транзакции с помощью SELECT убедиться, что данные записались в БД.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO orders (employee_id, total_price, date_time)
-> VALUES (1, 0, NOW());
Query OK, 1 row affected (0.01 sec)

mysql> SET @last_order_id = LAST_INSERT_ID();
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@last_order_id, 1, 2);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@last_order_id, 2, 1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@last_order_id, 3, 3);
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE orders o
-> JOIN (
-> SELECT og.order_id, SUM(g.price * og.quantity) AS total_sum
-> FROM orders_goods og
-> JOIN goods g ON og.good_id = g.id
-> WHERE og.order_id = @last_order_id
-> GROUP BY og.order_id
-> ) AS totals ON o.id = totals.order_id
-> SET o.total_price = totals.total_sum;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM orders WHERE id = @last_order_id;
+-----+
| id | employee_id | total_price | date_time |
+-----+
| 3977 | 1 | 345 | 2025-10-18 00:28:43 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM orders_goods WHERE order_id = @last_order_id;
+-----+
| order_id | good_id | quantity |
+-----+
| 3977 | 1 | 2 |
| 3977 | 2 | 1 |
| 3977 | 3 | 3 |
+-----+
3 rows in set (0.00 sec)

mysql>
```

- Написать транзакцию в ходе которой происходит ошибка. По аналогии с предыдущим заданием использовать таблицы orders, orders_goods. Последняя команда вставки должна завершаться ошибкой (например, попытаться создать запись в orders_goods, в которой указан несуществующий товар). Выполнить SELECT после транзакции и убедиться, что ни одна запись, входящая в транзакцию не появилась в БД.

Тут важно заметить, что вся транзакция не отвалилась, как я понял, в MySQL ошибка внутри транзакции не вызывает автоматический ROLLBACK всей транзакции, откатывается только неудавшаяся операция, а не весь блок. Ну и собственно валидные товары в транзакции прошли, не валидные не прошли.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO orders (employee_id, total_price, date_time)
-> VALUES (1, 0, NOW());
Query OK, 1 row affected (0.00 sec)

mysql> SET @bad_order_id = LAST_INSERT_ID();
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@bad_order_id, 1, 1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@bad_order_id, 2, 2);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@bad_order_id, 9999, 1);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('fast_food_resto_1032259220'.`orders_goods`, CONSTRAINT `orders_goods_ibfk_2` FOREIGN KEY (`good_id`) REFERENCES `goods` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE)
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM orders WHERE id = @bad_order_id;
+----+-----+-----+-----+
| id | employee_id | total_price | date_time |
+----+-----+-----+-----+
| 3978 | 1 | 0 | 2025-10-18 00:30:54 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM orders_goods WHERE order_id = @bad_order_id;
+-----+-----+-----+
| order_id | good_id | quantity |
+-----+-----+-----+
| 3978 | 1 | 1 |
| 3978 | 2 | 2 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

- Задание на проверку влияния установленного уровня изоляции транзакций. Для выполнения задания потребуется открыть еще один терминал для работы с sql. В первом терминале начать выполнение транзакции (можно использовать те же таблицы, что и в предыдущем задании). Вставить запись в orders и связанные с ней записи в orders_goods. В конце транзакции сделать sleep на 9999 секунд (сочетание SELECT и встроенной функции sleep(time)). Данное действие будет являться имитацией длительной транзакции.

Во втором терминале выполнить установку уровня изоляции READ_UNCOMMITTED. Выполнить select по тем данным, которые вставляли в первом терминале. Убедиться, что данные возвращаются.

Далее во втором терминале выполнить установку уровня изоляции READ_COMMITTED или выше. Выполнить select по тем данным, которые вставляли в первом терминале. Убедиться, что данные **не** возвращаются, т.к. были еще не закоммичены в транзакции.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO orders (employee_id, total_price, date_time)
-> VALUES (1, 0, NOW());
Query OK, 1 row affected (0.00 sec)

mysql> SET @test_order_id = LAST_INSERT_ID();
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT LAST_INSERT_ID();
+-----+
| LAST_INSERT_ID() |
+-----+
|          3980 |
+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO orders_goods (order_id, good_id, quantity) VALUES (@test_order_id, 1, 1);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT SLEEP(9999);
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> SET @test_order_id = 3980;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM orders WHERE id = @test_order_id;
t_order_id;
+-----+-----+-----+-----+
| id | employee_id | total_price | date_time |
+-----+-----+-----+-----+
| 3980 | 1 | 0 | 2025-10-18 00:42:09 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM orders_goods WHERE order_id = @test_order_id;
+-----+-----+-----+
| order_id | good_id | quantity |
+-----+-----+-----+
| 3980 | 1 | 1 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM orders WHERE id = @test_order_id;
Empty set (0.00 sec)

mysql> |
```